MVP BUILD PLAYBOOK

AI Procurement Risk Intelligence Build Guide

A step-by-step engineering playbook for three connected AI products: Supplier Risk Scoring, Procurement Fraud & Anomaly Detection, and the unifying Procurement Risk OS — including recommended tech stack and build-order modules for each.

Supplier Risk Scoring | Fraud & Anomaly Detection | Procurement Risk OS | Hackathon-ready

Scope
3 product blueprints, unified architecture

Format
Tech stack + module-by-module build order

Recommended path
Build #3, powered by #1 + #2

EXECUTIVE SUMMARY

Recommended Build Strategy

All three ideas share the same data foundation (suppliers, purchase orders, invoices, contracts, payments) and the same AI layer (ML models + LLM reasoning). The fastest path to a compelling, working product is to build the **AI Procurement Risk OS** as the primary product — an autonomous operational layer that runs in the background — and implement Supplier Risk Scoring and Fraud & Anomaly Detection as the internal engines/services that power it.

Shared Data Layer → **#1 Risk Scoring Engine** + **#2 Fraud Detection Engine** → **#3 Risk OS (Command Center + Autonomous Actions)**

Why this order: Building #1 and #2 first gives you real, structured outputs (risk scores, flagged anomalies) to drive #3's automated workflow. This means the Risk OS never has to "make up" a trigger condition — it reacts to real engine outputs, evaluates every new PO automatically, and lets an LLM turn the result into alerts, memos, and recommended actions. It also lets you demo three products for the price of one build.

How to read this playbook

- Each idea has its own section: **Overview** → **Tech Stack** → **Build Modules (in build order)**.
- Modules are meant to be built and tested sequentially — each one produces a working, demoable increment.
- Section 3 (the Risk OS) assumes Sections 1 and 2 are already built as callable services/functions.
- A suggested hackathon timeline and demo script are included at the end.

Shared Foundation (build this once, use everywhere)

All three ideas can run on one shared backbone. Build this first regardless of which idea you start with.

LAYER	RECOMMENDED TECH	WHY
Data storage	PostgreSQL (Supabase or Neon for hackathon speed)	Relational data (suppliers, POs, invoices, contracts) with fast hosted setup and built-in auth/API
Backend / API	Python + FastAPI	Fast to build, async, plays well with ML/LLM libraries, auto-generated docs
Data processing	pandas, numpy	Cleaning and feature engineering for scoring, anomaly detection
ML models	scikit-learn (Isolation Forest, Logistic Regression, Gradient Boosting)	Lightweight, no GPU needed, trains fast on sample/ synthetic data
LLM layer	Anthropic Claude API (Claude Sonnet) via API or LangChain	Explanations, summarization, natural-language Q&A, contract analysis
Vector search (for contracts)	pgvector or ChromaDB	Semantic search over contract text and supplier notes
Frontend	React + Vite + Tailwind CSS, Recharts for charts	Fast to build a clean live dashboard UI
Background jobs / event trigger	Python background task (FastAPI BackgroundTasks / a simple polling worker), or Celery + Redis if time allows	Lets new PO events trigger risk evaluation automatically, without a user asking
Notifications	Slack Incoming Webhooks, SMTP (or a mocked notification log) for email	Real-time alerts pushed to where procurement managers already work

Auth (optional for demo)	Supabase Auth or skip for hackathon	Only needed if multi-user demo required
Hosting	Vercel (frontend), Render/Railway (backend + worker), Supabase (DB)	Free tiers, fast deploy, good for live demo links
Sample data	Synthetic dataset generator (Python + Faker) or Kaggle procurement datasets	Real procurement data is rarely available — simulate realistic suppliers/POs/invoices

Hackathon tip: Don't wait for "real" data. Spend 30–45 minutes writing a synthetic data generator (suppliers, POs, invoices, payments, contracts) with deliberately planted anomalies and risk patterns. This makes your demo controllable and your AI outputs impressive on cue.

01

AI Supplier Risk Scoring System

Evaluates suppliers across delivery, quality, price, financial, contract, and external-news signals to produce a dynamic risk score with explanations.

Output Contract

Risk Score (0–100) | **Risk Tier (Low/Medium/High/Critical)** | **Key Risk Factors** | **AI Recommendations**

Idea-Specific Tech Stack

COMPONENT	TECH CHOICE
Feature engineering	pandas — rolling averages for delivery delays, price volatility (std dev), quality defect rates
Scoring model	Weighted rule-based score for MVP speed, upgradeable to scikit-learn Gradient Boosting / Logistic Regression once labeled data exists
Financial signal (optional stretch)	Public financial health proxy — credit-rating style heuristics, or mocked "financial health index" field
External news/events	News API (NewsAPI.org) or web search + Claude for sentiment/event extraction (e.g. bankruptcy, lawsuits, factory fire)
Explanation layer	Claude API — turns the numeric score + top contributing factors into a plain-English narrative and recommendation
Storage	PostgreSQL tables: suppliers, deliveries, quality_checks, price_history, contracts, risk_scores

Build Modules (in order)

1 Data Model & Synthetic Dataset — ~1–2 hrs

Goal: have realistic supplier data to score against.

- Design tables: suppliers, delivery_records, quality_records, price_history, financial_indicators, contract_violations, news_events.
- Generate 30–50 synthetic suppliers with realistic variance; plant 5–8 "obviously risky" suppliers for demo purposes.
- Seed the database via a Python script.

Deliverable: populated database + ERD

2 Feature Engineering Pipeline — ~2 hrs

Goal: convert raw records into per-supplier signal features.

- Delivery performance → on-time %, average delay days, trend over last N orders.
- Quality → defect rate, return rate, complaint count.
- Price → % price change over time, volatility vs. category average.
- Financial → mocked/heuristic financial health index.
- Contract → number/severity of violations in last 12 months.
- External → news sentiment score (-1 to +1) and flagged event types.

Deliverable: `compute_supplier_features(supplier_id)` function returning a feature dict

3 Risk Scoring Model

~2–3 hrs

Goal: turn features into one 0–100 score with tier classification.

- MVP: weighted sum (e.g. delivery 25%, quality 20%, price volatility 15%, financial 20%, contract violations 10%, news/events 10%) — tune weights, normalize each feature to 0–100 first.
- Stretch: replace weights with a trained scikit-learn model once you have labeled/simulated "actually failed" suppliers.
- Map score to tiers: 0–39 Low, 40–59 Medium, 60–79 High, 80–100 Critical.

Deliverable: `score_supplier(supplier_id) → {score, tier, factor_breakdown}`

4 AI Explanation & Recommendation Layer

~2 hrs

Goal: turn numbers into a decision-ready narrative.

- Prompt Claude with the supplier's score, tier, and top 3 contributing factors (with actual numbers).
- Ask for: a 2–3 sentence risk summary, the key drivers in plain English, and 2–3 concrete recommended actions (e.g. "request updated financials," "diversify supply," "renegotiate SLA").
- Cache responses per supplier to avoid recomputation on every page load.

Deliverable: `explain_risk(supplier_id) → narrative + recommendations`

5 API & Dashboard UI

~2–3 hrs

Goal: make the score visible and explorable.

- FastAPI endpoints: `GET /suppliers`, `GET /suppliers/{id}/risk`.
- React dashboard: sortable supplier table with risk tier color badges, a supplier detail view with a radar/bar chart of factor breakdown, and the AI narrative + recommendations.

Deliverable: working risk dashboard, deployable and demoable

02 AI Procurement Fraud & Anomaly Detector

Analyzes purchase orders, invoices, supplier data, and payments to flag suspicious or unusual activity, with LLM-generated explanations for human review.

Output Contract

- **Flagged Transaction**
- **Anomaly Type**
- **Severity Score**
- **"Why flagged" explanation**

Idea-Specific Tech Stack

COMPONENT	TECH CHOICE
Statistical analysis	numpy/scipy — z-scores, IQR outlier detection on invoice amounts, quantities, unit prices
Anomaly detection (unsupervised)	scikit-learn Isolation Forest and/or Local Outlier Factor on multi-feature transaction vectors
Clustering	scikit-learn KMeans / DBSCAN — group similar transactions/suppliers to spot outlier clusters (e.g. split invoices, duplicate payments)
Rule engine (MVP fast-path)	Hand-written rules: duplicate invoice numbers, round-number invoices, weekend/after-hours POs, amounts just under approval thresholds, new supplier + high value
LLM explanation layer	Claude API — converts flagged pattern + supporting numbers into a human-readable "why this needs review" note
Storage	PostgreSQL tables: purchase_orders, invoices, payments, anomaly_flags

Build Modules (in order)

1 Transaction Data Model & Synthetic Dataset — ~1–2 hrs

Goal: realistic transaction volume with a known set of planted fraud patterns.

- Tables: purchase_orders, invoices, payments, linked to suppliers.
- Generate hundreds of "normal" transactions, then deliberately plant 10–15 anomalies: duplicate invoices, invoice-splitting (multiple invoices just under approval limit), price spikes vs. historical average, payments to a supplier not in the approved PO, weekend/odd-hour submissions.

Deliverable: seeded transactions table with a hidden "ground truth" fraud label column (used only for your own evaluation, not shown to the model)

2 Rule-Based Fast Flags — ~1.5 hrs

Goal: get obvious, explainable flags working immediately (great early demo win).

- Implement checks: duplicate invoice numbers/amounts, round-number bias, same-day multiple POs to one supplier, invoice amount just under an approval threshold, new/unverified supplier with high-value order.
- Each rule outputs: flag type, severity (Low/Med/High), and the specific field values that triggered it.

Deliverable: `run_rule_checks(transaction) → list of flags`

3 Statistical & Model-Based Anomaly Detection

~2–3 hrs

Goal: catch subtler anomalies rules can't hard-code.

- Build a numeric feature vector per transaction: amount, deviation from supplier's historical average price, quantity, time-of-day/day-of-week encoding, days between PO and invoice.
- Compute z-scores/IQR outliers per feature for a quick statistical pass.
- Train an Isolation Forest on the full feature set for a combined anomaly score; use KMeans/DBSCAN to detect outlier clusters of suppliers/transactions.

Deliverable: `anomaly_score(transaction) → 0–1 score + contributing features`

4 Flag Aggregation & Severity Scoring

~1 hr

Goal: combine rule flags + model anomaly score into one review priority.

- Merge rule-based flags and statistical/model score into a single severity score (e.g. weighted combination, or "any Critical rule = High severity regardless of model score").
- Store results in an anomaly_flags table for the review queue.

Deliverable: `evaluate_transaction(id) → {severity, flags[], anomaly_score}`

5 LLM Explanation Layer + Review UI

~2–3 hrs

Goal: make flags reviewable in plain English, not raw numbers.

- Prompt Claude with the transaction details + triggered flags + anomaly features → generate a short "why this was flagged" explanation and a suggested next action (approve, request documentation, escalate).
- Build a review queue UI: sortable by severity, filterable by flag type, with an approve/dismiss/escalate action (state only, no need for real workflow in MVP).

Deliverable: working fraud review dashboard

03 AI Procurement Risk OS — Recommended Main Build

An autonomous operational layer that runs in the background of procurement — continuously monitoring activity, proactively surfacing risk, and taking or recommending action directly inside the workflow. Not a chatbot-first product; the chat/Q&A surface from the previous version is optional polish, not the core.

Positioning: An autonomous enterprise layer that continuously monitors procurement operations, detects supplier and transaction risk, and takes or recommends corrective action before those risks become costly problems. It combines Supplier Risk Scoring (#1) + Fraud Detection (#2) + workflow automation into one enterprise product — the intelligence lives in the background, and the UI is a command center for oversight and one-click action, not a search box.

Core Interface: Proactive Command Center

The dashboard is the product. It should always be "live" — reflecting the current state of procurement risk without the manager having to ask anything.

- Live procurement pipeline
- Supplier risk matrix
- Real-time risk alerts
- Risk score + contributing factors
- Historical supplier trends
- One-click mitigation actions

Autonomous Workflow

New PO → AI evaluates supplier → Risk detected → Alert generated → Action triggered

Worked example: Acme Corp's risk score increases to 84/100 due to delivery delays and negative external events. The system can automatically: flag or hold the PO, request human approval, generate an audit report, draft a supplier inquiry, generate a risk mitigation memo, identify alternative suppliers, and send a Slack/email alert — without a manager having to ask a single question.

AI Architecture

Procurement Data → AI Risk Engine → Business Integration Layer → Risk Dashboard & Automated Actions

- **ML / Heuristics** — numerical supplier risk scoring (Idea #1's engine).
- **LLM** — risk explanations and document generation (memos, audit reports, supplier inquiries).
- **Anomaly Detection** — fraud and unusual transaction detection (Idea #2's engine).

Enterprise Integration (real or simulated for hackathon)

SYSTEM	HACKATHON APPROACH
SAP / ERP	Simulate with a mock REST endpoint / webhook receiver that accepts "new PO" events in an SAP-like payload shape — real SAP integration is out of scope for an MVP
Purchase orders	Internal table + a `POST /events/new-po` webhook that any external system (or your own simulator script) can call to trigger the pipeline
Invoices / Accounts payable	Internal tables, feeding Idea #2's fraud engine directly
Supplier databases	Internal suppliers table (same one used by Idea #1); treat as the "master data" the OS enriches with live risk scores
Slack / Email	Slack Incoming Webhook for real alerts (fast to set up); email via SMTP or a mocked "sent notifications" log if time is short

Idea-Specific Tech Stack

COMPONENT	TECH CHOICE
Event ingestion	FastAPI webhook endpoint `POST /events/new-po` (or a scheduled poller) that represents "a new PO/ invoice entered the system"
Risk Engine	Direct function calls into Idea #1 (`score_supplier`) and Idea #2 (`evaluate_transaction`) — no chat/ LLM router needed, the pipeline calls them automatically
Decision / rules layer	Simple threshold + policy rules (e.g. score >= 80 → hold PO + alert; score 60–79 → alert only; anomaly severity High → flag for review) — a lightweight rules table, editable without redeploying
Action layer (document generation)	Claude API — generates audit reports, supplier inquiry drafts, and risk mitigation memos from structured risk data; each action type is a distinct prompt template
Alternative supplier lookup	Simple similarity query on suppliers table (same category/region, lower risk score) — no ML needed for MVP
Notifications	Slack Incoming Webhook for real-time alerts; SMTP or logged notification record for email
Background execution	FastAPI BackgroundTasks for the hackathon (Celery + Redis if you want true async workers)
Audit trail	An `actions_log` table recording every automated/triggered action, timestamp, and status (proposed / approved / executed) — this is what makes the "autonomous" claim credible in a demo
Command Center UI	React dashboard: pipeline view, supplier risk matrix (scatter/heatmap), live alert feed, action buttons wired to the actions API

Build Modules (in order)

1 Wrap Engines #1 & #2 as the Core Risk Engine — ~1 hr

Goal: expose existing risk-scoring and fraud-detection logic as clean, callable functions the pipeline can invoke automatically.

- Wrap Idea #1's `score_supplier`/`explain_risk` and Idea #2's `evaluate_transaction` as internal functions with typed inputs/outputs.
- Add convenience aggregate functions: `get_high_risk_suppliers(tier)`, `get_anomalous_transactions(min_severity)` — these power the dashboard, not a chat query.

Deliverable: a "risk_engine" module the event pipeline can call directly

2 Event Ingestion & Autonomous Trigger — ~1.5 hrs

Goal: implement New PO → AI evaluates supplier automatically, with no user action required.

- Build `POST /events/new-po` — accepts a PO payload, stores it, and immediately calls the Risk Engine (Module 1) for the associated supplier and the PO/invoice itself.
- Write a small "ERP simulator" script that posts a stream of PO events (including a few deliberately high-risk ones) so the pipeline can be demoed without a real SAP connection.

Deliverable: posting a PO event automatically produces a fresh risk score and anomaly evaluation, with zero manual steps

3 Decision Rules & Alert Generation

~1.5 hrs

Goal: turn a risk score/anomaly flag into a concrete alert, per policy.

- Define a small policy table: score/severity thresholds → alert level (Info / Warning / Critical) and default recommended action.
- On every evaluation from Module 2, check policy and, if triggered, create an alert record (supplier/transaction, reason, severity, recommended action, timestamp).

Deliverable: `alerts` table populating automatically as risky POs come in

4 Automated Action Layer (LLM Document Generation)

~3 hrs

Goal: implement the concrete one-click actions the OS can take or draft.

- Build discrete action functions, each backed by a Claude prompt template using the supplier's risk data: `flag_or_hold_po()`, `request_human_approval()`, `generate_audit_report()`, `draft_supplier_inquiry()`, `generate_mitigation_memo()`, `find_alternative_suppliers()`.
- Every action call writes a row to the `actions_log` table (proposed → approved/executed) so the audit trail is real, not just a UI toast.
- `find_alternative_suppliers()` is a simple query (same category, lower risk score) — no LLM needed, keep it fast and deterministic.

Deliverable: every action listed in the worked example is independently callable and logged

5 Notification Layer (Slack / Email)

~1 hr

Goal: get alerts out of the dashboard and into where managers actually work.

- Wire a Slack Incoming Webhook to fire on Critical/Warning alerts from Module 3, formatted with supplier name, score, top factors, and a link to the dashboard.
- Add email (SMTP) as a second channel, or a mocked "notifications sent" log entry if SMTP setup risks demo time.

Deliverable: a real Slack message appears within seconds of a risky PO being posted

6 Command Center Dashboard

~3–4 hrs

Goal: build the proactive, always-live interface — this is the product's face.

- **Live pipeline view** — recent POs streaming in with live risk evaluation status.
- **Supplier risk matrix** — scatter/heatmap of suppliers by risk score and spend/volume, color-coded by tier.
- **Real-time alert feed** — newest alerts first, severity-coded, with the recommended action visible inline.
- **Supplier detail view** — risk score, contributing factor breakdown, historical trend chart over time.
- **One-click mitigation actions** — buttons wired directly to Module 4's action functions (hold PO, draft inquiry, generate memo, find alternatives), each confirming and logging the result.

Deliverable: full Command Center, updating live as simulated PO events arrive

7 Polish & Demo Hardening (stretch) ~1–2 hrs

Goal: de-risk the live demo of an autonomous, event-driven system.

- Script a clean sequence of PO events (including the Acme Corp-style example) so the live pipeline demo is reliable and repeatable on stage.
- Add graceful fallbacks if an action call fails (never let a button spin forever — show a clear error state).
- Rehearse: trigger a new PO live, show the alert appear on the dashboard and in Slack within seconds, then trigger one mitigation action end-to-end.

Deliverable: demo-safe, rehearsed Procurement Risk OS

EXECUTION PLAN

Suggested Hackathon Timeline (~24–30 hrs)

Built around delivering the Risk OS's live, autonomous pipeline as the headline demo, with #1 and #2 as its engines.

TIME BLOCK	FOCUS	KEY OUTCOMES
Hour 0–2	Shared foundation	DB schema, synthetic data generator, repo/deploy skeleton (FastAPI + React) running end-to-end with a "hello world" route
Hour 2–7	Idea #1 — Supplier Risk Scoring	Feature engineering → weighted scoring model → Claude explanation layer → basic risk dashboard
Hour 7–13	Idea #2 — Fraud & Anomaly Detection	Rule-based flags → Isolation Forest/clustering anomaly score → severity aggregation → review queue UI
Hour 13–15	Event ingestion + decision rules	`POST /events/new-po` webhook, ERP simulator script, policy thresholds → automatic alert generation
Hour 15–19	Automated action layer	LLM-backed action functions (hold PO, audit report, supplier inquiry, mitigation memo, alternative suppliers) + actions_log audit trail
Hour 19–20	Notifications	Slack webhook firing on Critical/Warning alerts, email/log fallback
Hour 20–26	Command Center dashboard	Live pipeline, supplier risk matrix, real-time alert feed, one-click mitigation buttons
Hour 26–28	Polish & rehearse	Script a clean PO event sequence, fix edge cases, rehearse the full demo script twice
Hour 28–30	Buffer	Deployment issues, last-minute bug fixes, slides

If time is short: Cut Idea #2's clustering/DBSCAN model and rely on Isolation Forest + rules only. Cut the email channel and Slack-only for notifications. Reduce the action layer to two or three actions (e.g. flag/hold PO + generate mitigation memo) rather than all six — a smaller set that works live beats a longer list that doesn't.

Demo Script (5 minutes)

- **0:00–0:45** — Frame the problem: procurement teams can't manually monitor supplier risk and unusual transactions across hundreds of POs in real time — by the time someone notices, the PO is already approved.
- **0:45–1:30** — Open the Command Center dashboard: show the live pipeline and supplier risk matrix at rest, all green/low-risk.
- **1:30–2:30** — Trigger a live PO event for a supplier like "Acme Corp" (via the ERP simulator). Watch the dashboard update in real time: risk score jumps to 84/100, a Critical alert appears in the feed, and a Slack message lands seconds later — with zero manual input.
- **2:30–3:30** — Open the alert: show the contributing factors (delivery delays, negative external events) and the recommended action already staged.
- **3:30–4:30** — Click one-click mitigation actions live: hold the PO, generate a risk mitigation memo, and show 2–3 alternative suppliers surfaced automatically.
- **4:30–5:00** — Close on the audit trail (actions_log) proving every step was logged, then recap the architecture in one slide: Procurement Data → AI Risk Engine → Business Integration Layer → Dashboard & Automated Actions.

Generated as an MVP build playbook. Tech choices favor speed-to-demo (hosted free tiers, lightweight scikit-learn models, one LLM provider) — swap in production-grade infrastructure (dedicated vector DB, labeled ML training data, workflow/approval systems, SSO) once past the prototype stage.